

INSTITUTO FEDERAL DO PIAUÍ

Curso de Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
Programação para Internet II, Prof. Rogério Silva, ADS IV – 2026.2

Atividade Escrita - 19/08/2026

Atividade 01 — Registrando o Atendimento

Disciplina: TEC.1052 — Programação para Internet II

Semana: 01 — Fundamentos de Aplicação Web

Formato: Dupla

Entrega: Repositório GitHub + Classroom

Prazo: até a véspera do próximo encontro

1. O que vocês já têm

O Mini-Prontuário, ao fim do Encontro 2, sabe fazer duas coisas: **listar pacientes** e **cadastrar pacientes**. Um prontuário eletrônico que só cadastra gente e nunca registra um atendimento não é, ainda, um prontuário — é uma agenda de contatos.

Esta atividade fecha essa lacuna: vocês vão construir o recurso **Encounter** (atendimento) de ponta a ponta, do banco de dados até a tela.

2. O que é um Encounter

Um atendimento tem:

Campo	Tipo	Obrigatório?	Exemplo
id	número	gerado pelo banco	1
patientId	número	sim, vem da URL	3
startedAt	texto (ISO)	sim	"2026-08-24T09:30"
chiefComplaint	texto	sim	"Cefaleia há três dias"
notes	texto	não	"Orientada hidratação e retorno em 7 dias."

chiefComplaint é a **queixa principal** — o motivo declarado da visita. notes é a conduta, o que ficou registrado depois do atendimento. É normal a conduta não existir ainda no momento em que o atendimento é aberto.

3. O contrato da API que vocês vão construir

GET /api/patients/:id/encounters → 200 [Encounter] | 404 se o paciente não existir

POST /api/patients/:id/encounters → 201 Encounter | 400 | 404

Reparem: a rota vive **dentro** do paciente. Isso não é acidente — um atendimento não existe sozinho, ele sempre pertence a alguém. O caminho da URL conta essa história.

4. Por níveis — trabalhem em ordem, sem pular

Esta atividade usa avaliação por **maestria**: você é avaliado pelo nível mais alto que alcançou de forma sólida, não por ter tentado tudo pela metade. É melhor entregar o Nível 1 completo e testado do que os três níveis pela metade.

Nível 1 — Essencial (*alcançar este nível já aprova a atividade*)

- ☐ Tabela `encounters` criada em `database/schema.sql`, com chave estrangeira para `patients`
- ☐ `GET /api/patients/:id/encounters` devolvendo os atendimentos daquele paciente
- ☐ `POST /api/patients/:id/encounters` validando `chiefComplaint` (obrigatório, não pode ser só espaços) e `startedAt` (formato `AAAA-MM-DDTHH:MM`)
- ☐ `GET/POST` devolvendo `404` quando o paciente não existir
- ☐ Todo SQL parametrizado — nenhuma concatenação de string em comando SQL
- ☐ `requests.http` com pelo menos um caso de sucesso e um caso de erro por rota

Dica de arranque: olhe o comentário `TODO ATIVIDADE 1` em `database/schema.sql`. Ele já diz exatamente quais colunas a tabela precisa ter.

Nível 2 — Proficiente

- ☐ Tela de detalhe do paciente, exibindo seus atendimentos (usando estado + render — nenhum fetch fora de `api.js`)
- ☐ Formulário de novo atendimento, mostrando o erro que vier do backend
- ☐ `404` tratado também no frontend (mensagem amigável, não uma tela quebrada)
- ☐ Componente `.encounter-item` em BEM, com tokens, testado em 375px de largura

Nível 3 — Avançado (*opcional — alimenta seu portfólio*)

- ☐ Estado de carregamento e estado de erro tratados separadamente na interface
- ☐ Atendimentos ordenados por data — decida no SQL ou no JavaScript, e **justifique a escolha no README**
- ☐ Contador de atendimentos no cartão do paciente, calculado (derivado), nunca guardado
- ☐ `README.md` com um diagrama do fluxo evento → ação → estado → render → API

5. Regras que continuam valendo

Escopo fechado: nenhuma biblioteca além de `express`, `better-sqlite3`, `tsx`, `typescript` e `Bootstrap` por CDN. Vale para vocês e para a IA que vocês usarem.

IA em modo chat, com protocolo: toda sugestão de código gerada por assistente passa pelo protocolo **V.E.R.M.** antes de entrar no projeto — Verifica, Explica, Reduz, Marca. Detalhes no *Guia de Bolso de IA*.

Você precisa saber explicar: qualquer linha do código entregue pode ser apontada, por mim ou pela sua dupla, e você precisa saber dizer o que ela faz e por quê.

6. Entregáveis obrigatórios

1. **Repositório GitHub** com histórico de commits de **ambos** os integrantes da dupla — commits só de uma pessoa indicam que a dupla não trabalhou junto.
2. **README.md** — como rodar o projeto, o que foi feito, qual nível foi alcançado.
3. **IA.md** — registro de toda interação relevante com assistente de IA (modelo no Guia de Bolso).
4. **Autoavaliação de 5 linhas** — o que travou durante a atividade e como vocês destravaram.

7. Perguntas frequentes

"A gente pode implementar o Nível 3 sem ter feito o Nível 2 direito?" Pode tentar, mas não recomendamos. A rubrica pesa a solidez de cada nível — um Nível 3 construído sobre um Nível 1 tremido vale menos do que um Nível 1 rocha sólida.

"E se a nossa dupla desfizer?" Fale comigo o quanto antes. Entregas individuais nesta atividade só acontecem em situação combinada previamente.

"Posso usar uma biblioteca de datas para formatar startedAt?" Não nesta semana. startedAt é um texto no formato AAAA-MM-DDTHH:MM — para exibir como 24/08/2026 09:30, um `split` e um `template string` bastam. Guardem a biblioteca de datas para quando ela resolver um problema real, não um problema de formatação simples.

"O que aconteceu com o campo notes se eu não digitar nada?" Ele deve ser salvo como `null`, não como `string` vazia. `null` significa "não existe". `String` vazia significaria "existe, e é nada" — são coisas diferentes.

Bom trabalho. Este é o primeiro recurso do prontuário que registra, de fato, um cuidado com um paciente — o resto até aqui foi cadastro.